Essence: A Personal Embedding Cluster Framework for Depth-Optimized Recommendations

Bidit Das
Independent Researcher | Dallas, TX
biditdas18@gmail.com

June 17, 2026

Abstract

We present **Essence**, a personal embedding cluster framework for recommendation that operates exclusively within a single user's engagement history, without consulting cross-user interaction data. Consumed items are embedded using a pre-trained sentence encoder and clustered into K semantic centroids via K-means; recommendations are generated by retrieving items similar to the centroid closest to the user's most recent activity. We evaluate Essence on two datasets, Last.fm-1K (music) and Amazon Books (e-commerce), against three baselines: a non-personalized popularity baseline, a collaborative filtering baseline (item-based k-nearest-neighbors over the user-item interaction matrix), and a content-based average-embedding baseline. Evaluation uses a chronological train/test split per user, consistent with the model's use of recent interaction history, and standard Recall@K. On long-tail items (those with a single training-set interaction), the non-personalized popularity baseline achieves exactly zero recall on both datasets, by construction, since globally popular items are never singletons. Item-based collaborative filtering recovers long-tail items only on the denser Amazon Books dataset, where it where its long-tail recall is comparable to the content baseline (overlapping 95% confidence intervals); on the sparser Last.fm-1K it recovers none under deterministic ranking, matching the popularity baseline. On both, its overall recall falls well below the content baseline. Essence achieves the highest long-tail recall of all systems on both datasets: Long-Tail Recall@10 of 0.0059 on Last.fm-1K (2.01× the content baseline) and 0.0197 on Amazon Books (1.59× the content baseline), while also achieving the highest overall Recall@10 among the personalized methods. On Last.fm-1K, an ablation on active cluster selection shows that replacing the recency-based mechanism with a static mean-embedding query collapses Essence's long-tail advantage, indicating the recency mechanism, not the clustering step alone, is responsible for the effect. We discuss Essence as a candidate-generation method suited to a dedicated discovery surface rather than a general-purpose ranking replacement. Code: `https://github.com/biditdas18/essence`

1 Introduction

Recommendation systems built on collaborative filtering (CF) derive their signal from cross-user interaction patterns: a user's future preferences are predicted from the behavior of users who interacted similarly in the past. This approach performs well on standard benchmarks, particularly on metrics dominated by popular items, but it carries a structural property worth examining directly: items with sparse interaction histories are, by construction, underrepresented in any signal derived from cross-user co-occurrence. A user with a narrow, idiosyncratic taste profile (engaging consistently with low-interaction items across a domain) receives recommendations shaped by aggregate similarity to other users, not by the specific combination of properties their own history reflects.

This work originated from a practical observation in music recommendation: users with precise,

narrow taste profiles are reliably steered toward broadly popular artists that are only loosely similar, while artists matching the specific texture of that taste remain unsurfaced regardless of how long the user searches the catalog manually. This pattern motivated the central question addressed here: can a recommendation signal be constructed using only a single user's own history, without any cross-user data, and if so, does it recover items that collaborative methods structurally underrepresent?

We propose Essence, a framework that embeds a user's consumed items into a semantic space using a pre-trained sentence encoder, clusters them into K interest profiles via K-means, and generates recommendations by retrieving items similar to the centroid closest to the user's recent activity. We refer to this as the *peer-isolation property*: no cross-user interaction graph is consulted at any stage. This is a deliberate architectural constraint, not an oversight: it is the mechanism under test. We describe the resulting objective as depth-optimized: the goal is to retrieve items deep in a user's long-tail interest space, rather than to maximize aggregate engagement on broadly popular items.

The contributions of this work are:

1. The **Essence architecture**: a personal recommendation framework requiring no collaborative signal and no supervised or cross-user recommender training, operating entirely on a single user's history.

2. A **recency-based active-cluster-selection** mechanism, evaluated against a static mean-embedding alternative via ablation (Section 5.3).

3. **Empirical evaluation** on two datasets: Last.fm-1K and Amazon Books, under a chronological train/test split, comparing Essence against a real collaborative filtering baseline (item-based KNN), a non-personalized popularity baseline, and a content-based average-embedding baseline.

4. An empirical finding across two datasets: non-personalized popularity achieves exactly zero long-tail recall by construction; item-based collaborative filtering recovers long-tail items only on the denser dataset (none on the sparser one) and at a large cost to overall recall; and Essence achieves the highest long-tail recall of all systems on both.

2 Related Work

2.1 Collaborative Filtering

Matrix factorisation methods [5] and neural extensions [4] remain dominant in production recommendation. Their principal limitation is popularity bias: items with sparse interactions are poorly represented, resulting in near-zero recall for long-tail items. CF is optimised for the median user: it performs well on what is broadly consumed and comparatively weakly on what is individually resonant.

2.2 Multi-Interest Retrieval

MIND [1] introduced dynamic routing to extract multiple interest capsules from a user's interaction history, explicitly addressing the failure of a single user vector to represent diverse tastes. ComiRec [2] extended this with explicit diversity controls at serving time.

Essence shares the multi-interest intuition with MIND and ComiRec, the recognition that a user's preferences cannot be collapsed into a single vector. However, the similarity ends at the surface. The distinction operates at two levels.

Data level. MIND and ComiRec require population-level interaction graphs. Their dynamic routing is trained on millions of (user, item, interaction) triples across the full user base. Cross-user behavioural data is not optional: it is the mechanism by which interest capsules are learned. Essence requires no data from any other user, ever. The K-means centroids are derived entirely from one user's own history.

Representation level. Because MIND's item embeddings are trained on co-occurrence patterns across users, what an item *means* in that space is partly defined by who else liked it. An obscure track heard by very few users is underrepresented or invisible in that embedding space, as its co-occurrence signal is too sparse to be learned reliably. Essence uses a pre-trained sentence encoder on item metadata only. The embedding captures metadata-derived semantic information about the item, independent of how many people have interacted with it. A singleton item can be embedded as readily as a chart-topping one; the quality of that representation depends on how informative the item's metadata is, not on its interaction count.

This distinction suggests a hypothesis: Essence should recover long-tail items at higher rates than systems whose embeddings are shaped by co-occurrence sparsity. We do not test this directly against MIND or ComiRec in this work; that comparison is left to future work (Section 7).

Table 1 summarises the architectural distinctions between Essence, MIND, ComiRec, and standard CF.

Table 1: Architectural comparison: Essence vs. prior systems. The primary distinction is not the clustering mechanism, it is the data requirement and what each system is optimised to find. The MIND and ComiRec entries reflect architectural expectations from their reliance on cross-user co-occurrence; these two methods are not empirically evaluated in this work (Section 5 evaluates CF, Content, and Essence). A dash (—) in the long-tail item recovery row marks MIND and ComiRec as not measured in this work.

Property	**CF**	**MIND**	**ComiRec**	**Essence**
Cross-user data required	Yes	Yes	Yes	**No**
Multiple interest profiles	No	Yes	Yes	**Yes**
Model training required	Yes	Yes	Yes	**No**
Long-tail item recovery	Limited	—	—	**Yes**
No cross-user data required	No	No	No	**Yes**
Designed to surface niche items	No	No	No	**Yes**
Embedding source	Co-occurrence	Co-occurrence	Co-occurrence	**Item metadata**

2.3 Session-Based and Content-Based Methods

BERT4Rec [3] and GRU4Rec [6] model sequential user behaviour using transformer and recurrent architectures, capturing short-term context effectively. Content-based filtering relies on item feature similarity without interaction logs but typically averages preferences into a single vector, losing intra-user diversity.

Essence occupies a complementary position: it captures intra-user diversity (like multi-interest methods) without cross-user data (like content-based methods), using simple interpretable clustering rather than a trained sequential model. We acknowledge that BERT4Rec offers richer sequential modelling in high-interaction settings; Essence targets privacy-first, low-infrastructure deployment contexts where such models are not viable.

3 The Essence Framework

3.1 Notation

Table 2 defines all mathematical notation used in this section. For each symbol a plain-English equivalent is provided.

Table 2: Notation Legend

Symbol	Plain-English Meaning
H_u	All items user u has consumed (full history)
$i_1, i_2, \ldots$	Individual items in the user's history, in order
$\phi(i)$	Embedding of item i, a 384-dim vector describing its content
E_u	All embeddings stacked into a matrix, one row per item
K	Number of interest clusters (we use $K = 3$)
c_1, c_2, c_3	The K cluster centres, one per interest profile
c_k^*	Active cluster centre, closest to the user's recent activity
$\mu(\cdot)$	Mean (average) of a set of vectors
$\cos(\mathbf{a}, \mathbf{b})$	Cosine similarity: how alike two vectors are
$I \setminus H_u$	All items the user has *not* yet consumed, the candidate pool
R_u	Final ranked recommendation list for user u
M	Number of recommendations to return (we use $M = 10$)

3.2 Problem Formulation

Given a user's history H_u (all items they have consumed), the goal is to produce a ranked list $R_u \subseteq I \setminus H_u$ of items they have not yet consumed but are likely to enjoy. Essence accomplishes this using only H_u and a shared embedding function $\phi : I \to \mathbb{R}^d$. No data from any other user is used at any stage.

3.3 Item Embedding

Each item is described by a short text string encoding its content properties. For music: track name and artist name. For books: title and author name. We convert this text into a 384-dimensional vector using the pre-trained sentence encoder `all-MiniLM-L6-v2` [8, 7]. Items that are semantically similar produce similar vectors; items that are stylistically distant produce different vectors. Critically, this embedding is derived from item metadata text, not from interaction patterns. A niche item can be embedded as readily as a popular one; representation quality depends on the informativeness of the metadata rather than on interaction frequency. All embeddings are computed once offline and cached.

Formally, $\phi(i)$ is the embedding of item i. For user u with history $H_u = \{i_1, \ldots, i_n\}$ we compute the embedding matrix $E_u \in \mathbb{R}^{n \times d}$, where each row is the embedding of one item.

3.4 Interest Profile Clustering

We apply K-means clustering to E_u with $K = 3$, yielding centroids $\{c_1, c_2, c_3\}$. Each centroid represents a distinct interest profile: the semantic centre of gravity of one coherent facet of the user's taste. A user who consumes jazz, indie rock, and ambient electronica might end up with one centroid per genre. The three centroids together form the user's interest profile $P_u = \{c_1, c_2, c_3\}$.

This is the core differentiator from average-embedding approaches. The mean of all embeddings collapses all interests into a single blurred point that sits between clusters and represents none of them well. K-means preserves the separation between interest modes, enabling targeted retrieval against each mode independently.

$K = 3$ was selected as the minimum number of centroids capable of representing the tripartite interest structure commonly observed in consumption behaviour (e.g., mood-based, genre-based, and era-based clusters). Sensitivity analysis over $K \in \{2, 3, 4, 5\}$ is a planned extension.

For users with fewer than K items, Essence falls back to the content baseline (single mean embedding).

3.5 Active Cluster Selection

At inference time, Essence selects the active centroid c_k^* as the cluster centre closest to the mean embedding of the user's r most recent training-set interactions, ordered chronologically (default $r = 10$):

$$c_k^* \;=\; \arg\min_k \left\| c_k - \mu(\phi(h_{n-r}), \ldots, \phi(h_n)) \right\|_2 \tag{1}$$

This mechanism assumes recent activity is informative about which interest cluster is currently active. We test this assumption directly via ablation (Section 5.3): replacing the recency-based query with the mean embedding of the user's entire training history degrades long-tail recall substantially on Last.fm-1K, indicating that recency carries signal beyond what the K-means clustering step alone provides. The recency mechanism is retained as the default not as an unexamined design choice but because the ablation supports it.

3.6 Recommendation Generation

Given the active centroid c_k^*, Essence retrieves the top-M unseen items by cosine similarity:

$$R_u \;=\; \text{top-}M \left\{ i \in I \setminus H_u \;:\; \cos(\phi(i),\, c_k^*) \right\} \tag{2}$$

This retrieval step operates entirely in embedding space with no collaborative signal. An item's probability of appearing in R_u is determined solely by its latent similarity to the user's current interest cluster, not by how many other users have interacted with it.

3.7 Architecture Overview

Figure 1 illustrates the full Essence pipeline from raw user history to final recommendations.

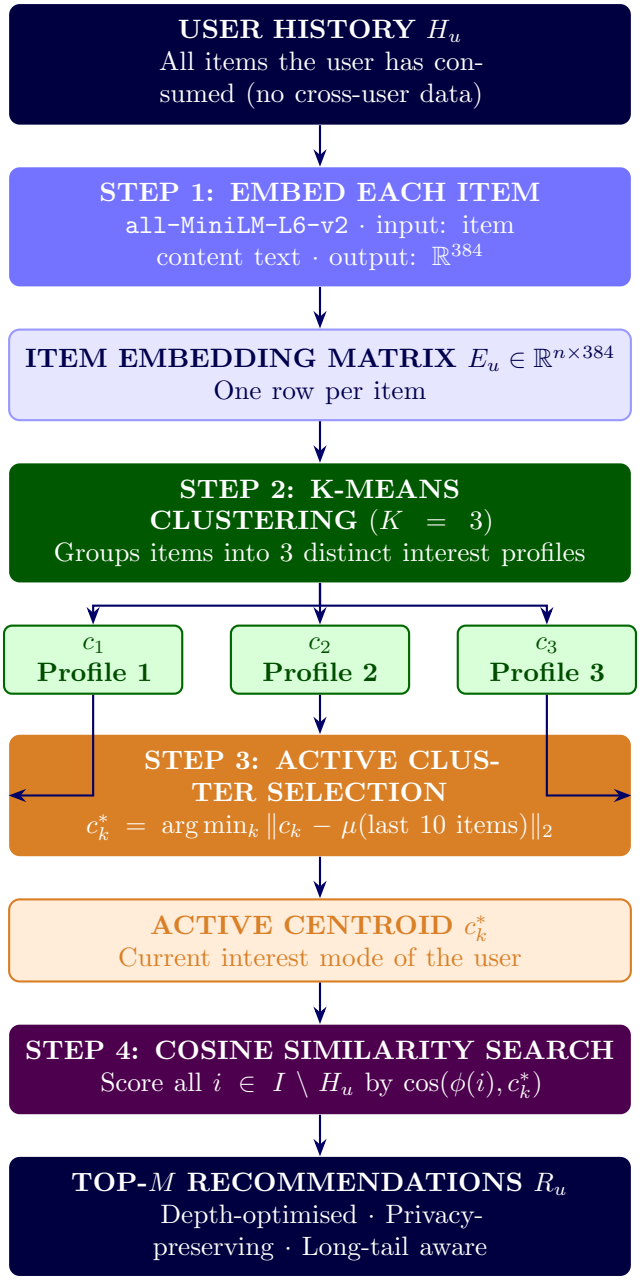

Figure 1: Essence recommendation pipeline. No cross-user data is consulted at any stage (peer-isolation property).

4 Experimental Setup

4.1 Datasets

Last.fm-1K (music). Contains listening histories from 992 users across approximately 19 million events [9]. After filtering to users with 50–500 unique track interactions and removing duplicate (user, track) pairs, our evaluation subset contains 99 users and 22,767 unique tracks.

Amazon Books (e-commerce). The Amazon Reviews 2023 Books 5-core dataset [10], which guarantees every user and item has at least 5 global interactions. After filtering to users with 20–200 unique item interactions and subsampling to 2,000 users (seed 42), our evaluation subset contains 2,000 users and 61,727 unique items. Item embeddings use title and author name, following the same metadata-only approach as Last.fm-1K.

4.2 Train/Test Split

For each user on both datasets, interactions are sorted chronologically and split per-user: the first 80% form the training set, and the final 20% form the test set. This chronological split is required for internal consistency with the active-cluster-selection mechanism (Section 3.5), which defines "recent" interactions in terms of training-set recency; a random split would make this definition incoherent, since a randomly held-out test item could chronologically precede items retained in training.

Item embeddings are computed once, offline, over the full item catalog (train ∪ test), so that no test item is excluded from the retrieval candidate pool due to missing embedding coverage. This is independent of the train/test split decision and applies identically under either splitting strategy.

4.3 Long-Tail Definition

Long-tail items are those with insufficient interaction signal for collaborative filtering to represent reliably. We define long-tail operationally per dataset, anchored to each dataset's interaction density:

- **Last.fm-1K.** Items with a training-set interaction count of exactly 1 (*singleton tracks*): 16,761 of the 18,679 items appearing in the training set (89.7%). The full candidate pool (train ∪ test) is 22,767 tracks.

- **Amazon Books.** Items with a training-set interaction count of exactly 1 (*singleton items*): 42,878 of the 50,924 items appearing in the training set (84.2%). The full candidate pool (train ∪ test) is 61,727 items. Despite the 5-core guarantee of at least 5 global interactions, subsampling to 2,000 users reduces most items to a single observed interaction in the subset's training split.

Generalisation principle. The long-tail threshold should be set at the point where collaborative filtering loses reliable signal, empirically where item interaction count falls below the minimum required for stable co-occurrence estimation. In both datasets evaluated here, the singleton threshold (train interaction count = 1) proved the most meaningful boundary: alternative thresholds either collapsed toward the full catalog (losing discriminative power) or excluded too few items. The key invariant is that long-tail items are defined on the *training set only*, after the train/test split, ensuring no leakage from held-out interactions into the long-tail classification.

LT-Recall@10 is computed only over test items falling in the long-tail set. Users with no long-tail test items are excluded from this metric.

4.4 Systems Compared

- **Random.** Recommends M items drawn uniformly from the unseen candidate pool. Establishes the floor against which all other systems are compared.

- **Popularity.** Recommends the M globally most-interacted-with items the user has not consumed. A non-personalized baseline; included for comparison but not treated as collaborative filtering.

- **CF (ItemKNN).** An item-based k-nearest-neighbors collaborative filtering baseline, computed via cosine similarity over the sparse user-item interaction matrix. Recommends items most similar, by co-occurrence pattern, to items the user has already consumed.

- **Content (Average Embedding).** Computes the mean of all the user's item embeddings and retrieves the M most similar unseen items by cosine similarity. Standard content-based

filtering without interest decomposition.

- **Essence ($K = 3$).** Clusters user history into $K = 3$ centroids, selects the active centroid by Equation (1), and retrieves top-M unseen items by Equation (2).

4.5 Metrics

What we are measuring. Each system produces a ranked list of 10 recommended items per user. We know which items that user actually consumed in the test set (ground truth). The evaluation asks: how much of that ground truth did the system recover?

This is a retrieval problem, not a classification problem. All systems are evaluated against the *full item catalog* with no negative sampling, following the full-ranking evaluation protocol [11]. Absolute recall values are low by construction under this protocol: the meaningful signal is relative comparison across systems evaluated under identical conditions.

- **Recall@10.** Fraction of the user's test items appearing in the top-10 recommendations.

$$\text{Recall@10} = \frac{|\, R_u \cap \text{Test}_u \,|}{|\, \text{Test}_u \,|}$$

  Example: 20 test items, 2 hits in top-10. Recall@10 = 0.10. Averaged across all users.

- **Long-Tail Recall@10 (LT-Recall@10).** Recall@10 restricted to singleton test items only.

$$\text{LT-Recall@10} = \frac{|\, R_u \cap \text{LT-Test}_u \,|}{|\, \text{LT-Test}_u \,|}$$

  A long-tail hit means the system recommended a niche item that this specific user genuinely consumed, without any popularity signal and without any other user's behaviour to guide it. This metric directly tests Essence's core hypothesis.

- **Lift over random.** Recall@10 divided by expected random recall ($M/|I|$). Normalises for candidate pool size and makes cross-dataset comparison interpretable.

4.6 Implementation

All systems implemented in Python 3.11. Embeddings via `sentence-transformers` (`all-MiniLM-L6-v2`, 384 dims). K-means via `scikit-learn` (`n_init=10`, `random_state=42`). All ranking steps break score ties deterministically (stable sort by item index), so reported results are exactly reproducible; this matters for sparse baselines such as CF (ItemKNN), where many candidate items receive identical scores. Confidence intervals for Recall@10 and LT-Recall@10 are obtained by bootstrapping over users (10,000 resamples, seed 42); LT-Recall intervals and per-user counts are computed over eligible users only (those with at least one long-tail test item). Code available at: https://github.com/biditdas18/essence

5 Results

5.1 Last.fm-1K

Table 3 reports results on 99 users (chronological 80/20 split per user). All systems rank against the full item catalog of 22,767 unique tracks (train ∪ test); 16,761 of these are long-tail (training-set singletons). Lift is computed against the theoretical random expectation (10/22,767 = 0.000439).

Table 3: Results on Last.fm-1K (99 users, chronological split, $M = 10$, $|I| = 22{,}767$). Lift computed against theoretical random expectation. Best non-random results in **bold**. LT/Content ratios are computed from unrounded recall values.

System	Recall@10	Lift	LT-Recall@10	LT / Content
Random	0.0001	—	0.0000	—
Popularity	0.0023	5×	0.0000	—
CF (ItemKNN)	0.0018	4×	0.0000	—
Content (Avg Embedding)	0.0038	9×	0.0029	1.0×
Essence ($K = 3$)	**0.0119**	**27×**	**0.0059**	**2.01×**

The Popularity baseline scores exactly zero long-tail recall, by construction, since globally popular items are never singletons. CF (ItemKNN) also recovers no long-tail items on Last.fm (LT-Recall = 0.0000): it assigns sparse singletons co-occurrence scores too weak to rank them reliably above the candidate field. This metric is sensitive to tie-breaking on sparse data: under non-deterministic ordering CF's long-tail recall fluctuates around 0.004, but resolves to zero once ties are broken deterministically (Section 4.6), indicating a weak and unstable long-tail signal rather than a reliably present one. Both Content (0.0029) and Essence (0.0059) recover long-tail items that CF does not. Essence's LT-Recall is 2.01× Content's, but on Last.fm-1K this comparison rests on long-tail hits for only 2 of 92 eligible users per method, and the bootstrap 95% CIs both include zero and overlap substantially (Table 5); we therefore treat the Last.fm-1K long-tail result as directional. The statistically substantive long-tail evidence is the 2,000-user Amazon Books evaluation (Table 4, Table 5). On overall Recall@10, Essence (0.0119) leads all personalized methods, with CF (0.0018) falling below even Content (0.0038); Essence is the only method that leads on both metrics.

5.2 Amazon Books

Table 4 reports results on 2,000 users (chronological 80/20 split per user, $|I| = 61{,}727$, 42,878 long-tail items). Lift is computed against theoretical random expectation ($10/61{,}727 = 0.00016$).

Table 4: Results on Amazon Books (2,000 users, chronological split, $M = 10$, $|I| = 61{,}727$). Lift computed against theoretical random expectation. Best non-random results in **bold**.

System	Recall@10	Lift	LT-Recall@10	LT / Content
Random	0.0001	—	0.0000	—
Popularity	0.0021	13×	0.0000	—
CF (ItemKNN)	0.0031	19×	0.0137	1.11×
Content (Avg Embedding)	0.0173	107×	0.0124	1.0×
Essence ($K = 3$)	**0.0221**	**136×**	**0.0197**	**1.59×**

The pattern holds: Essence outperforms Content on long-tail recall (0.0197, 95% CI $[0.0134, 0.0266]$, vs. 0.0124, $[0.0075, 0.0180]$; 1.59×), with each mean lying outside the other's 95% CI and a larger set of users contributing a long-tail hit (42 vs. 26 of 1,292 eligible users). Essence also leads overall Recall@10 (0.0221, $[0.0185, 0.0259]$, vs. Content 0.0173, $[0.0140, 0.0207]$), and Popularity scores zero. Unlike on the sparser Last.fm-1K, where CF recovered no long-tail items, here CF (ItemKNN) surfaces some: its long-tail recall (0.0137, 95% CI $[0.0084, 0.0196]$) is comparable to Content's (0.0124, $[0.0075, 0.0180]$) — the intervals overlap substantially, so the nominal 1.11× margin is not statistically meaningful — yet CF achieves this at a substantially lower overall

Recall@10 (0.0031 vs. 0.0173). On a denser dataset (2,000 users vs. 99), co-occurrence-based similarity recovers some low-interaction items (CF's long-tail CI clears zero, unlike on Last.fm-1K) but does not exceed a content baseline and pays a large cost in overall ranking precision — a profile distinct from Essence's, which improves on both metrics relative to Content.

Table 5: Bootstrap 95% confidence intervals and per-user support for Long-Tail Recall@10 (10,000 resamples over users, seed 42). Intervals and counts are over eligible users only (those with ≥ 1 long-tail test item). n_{elig}: eligible users; n_{hit}: those with at least one long-tail hit. Random and Popularity score exactly 0.0000 on both datasets by construction and are omitted.

Dataset	System	LT-Recall@10	95% CI	$\mathbf{n}_{\text{elig}}$	$\mathbf{n}_{\text{hit}}$
Last.fm-1K	CF (ItemKNN)	0.0000	[0.0000, 0.0000]	92	0
	Content (Avg Emb)	0.0029	[0.0000, 0.0078]	92	2
	Essence (K=3)	0.0059	[0.0000, 0.0172]	92	2
Amazon Books	CF (ItemKNN)	0.0137	[0.0084, 0.0196]	1,292	28
	Content (Avg Emb)	0.0124	[0.0075, 0.0180]	1,292	26
	Essence (K=3)	0.0197	[0.0134, 0.0266]	1,292	42

5.3 Active Cluster Selection Ablation

Section 3.5 motivates recency-based active cluster selection on the grounds that it captures session-aware context without requiring a trained sequential model. We test this directly by comparing it against an alternative: selecting the active centroid using the mean embedding of the user's entire training history, rather than the most recent $r = 10$ interactions.

Table 6: Active cluster selection ablation, Last.fm-1K (chronological split, $M = 10$).

Variant	Recall@10	LT-Recall@10	LT / Content
Essence (recency, $r = 10$)	0.0119	0.0059	2.01×
Essence (mean-select, full history)	0.0035	0.0005	0.16×
Content (Avg Embedding, reference)	0.0038	0.0029	1.0×

Mean-select performs below even the Content baseline on long-tail recall. This is consistent with the mechanism collapsing toward Content: selecting the centroid nearest a user's global mean embedding is functionally similar to querying with that mean directly, which is what Content already does, while incurring the additional cost of clustering without a corresponding benefit. Recency-based selection is retained as the default mechanism on the basis of this result, conducted on Last.fm-1K; we have not verified whether this finding generalizes to Amazon Books and note this as a scope limitation rather than an established cross-domain result.

5.4 Effect of User-Authored Review Embeddings (Amazon Books)

As a secondary experiment, we tested whether building user taste profiles from review text (first-person language describing a user's reaction to consumed items) rather than item metadata changes Essence's relative performance. Review coverage was complete for the 2,000-user Amazon Books subset.

Table 7: Amazon Books: metadata embeddings (Pass 1) vs. user-review embeddings (Pass 2). Pass 2 builds taste profiles from the user's own review text; candidates are scored using metadata embeddings.

Pass	System	Recall@10	LT-Recall@10	LT / Content
Pass 1 (metadata)	CF (ItemKNN)	0.0031	0.0137	—
	Content (Avg)	0.0173	0.0124	1.0×
	Essence ($K = 3$)	0.0221	0.0197	1.59×
Pass 2 (user review)	CF (ItemKNN)	0.0031	0.0137	—
	Content (Avg)	0.0027	0.0012	1.0×
	Essence ($K = 3$)	0.0041	0.0034	2.83×

Absolute recall degrades substantially in Pass 2. Two factors likely contribute. First, an input-distribution mismatch: taste profiles in Pass 2 are built from review-text embeddings, while candidate items are still scored using metadata embeddings. Although both are produced by the same encoder, review text and item metadata occupy different regions of that shared embedding space, and comparing across these regions is less discriminative than comparing inputs of the same type. Second, review text is a noisier input than structured metadata: a product review may describe shipping conditions or personal circumstances unrelated to the item's content, and an embedding built from off-topic text does not faithfully represent either the item or the user's response to it.

Essence's relative advantage over Content persists in Pass 2 (2.83× vs. 1.59× in Pass 1, both above parity), suggesting the K-means clustering mechanism extracts usable interest structure even under degraded input quality. We do not treat this as a strong claim, given the absolute recall values in Pass 2 are low enough that the comparison carries limited statistical weight; we report it as a directional observation and a motivation for the embedding-input-curation discussion in Section 7.

5.5 Cross-Dataset Pattern

Three findings hold consistently across both datasets and all conditions tested:

1. The non-personalized Popularity baseline achieves exactly zero long-tail recall in every configuration, by construction. Item-based CF (ItemKNN) recovers long-tail items only on the denser Amazon Books dataset (0.0137, comparable to the content baseline; CIs overlap, so the 1.11× is not significant); on the sparser Last.fm-1K it recovers none under deterministic ranking, matching Popularity. On both its overall Recall@10 is well below the content baseline, and it trails Essence on long-tail recall throughout. CF's long-tail recovery is therefore density-dependent, whereas Essence leads on long-tail recall on both datasets regardless of density.

2. Essence outperforms the content-based average-embedding baseline on long-tail recall in every primary configuration (Sections 5.1–5.2), and the ablation in Section 5.3 indicates this advantage depends specifically on recency-based active cluster selection rather than on K-means clustering alone.

3. Absolute recall values are low across all systems under full-catalog evaluation without negative sampling, a known property of this evaluation protocol [11] rather than evidence of system failure; the relative ordering between systems, evaluated under identical conditions, is the meaningful signal.

6 Discussion

6.1 Mechanism: Why Popularity-Driven Discovery Self-Reinforces

Collaborative filtering's reliance on cross-user co-occurrence creates a feedback property worth stating plainly: items that already have more interactions generate more training signal, which improves their representation in the model, which increases their likelihood of being recommended, which generates further interactions. Items with few interactions do not benefit from this loop, regardless of how well they might match any individual user's taste, because the signal needed to learn that match does not exist in sufficient quantity. This mechanism is clearest for the non-personalized popularity baseline, which records exactly zero long-tail recall in Section 5. Item-based CF escapes it only on the denser Amazon Books dataset, where co-occurrence signal suffices to surface some low-interaction items; on the sparser Last.fm-1K it too records zero long-tail recall under deterministic ranking, and on both it pays a large cost to overall recall. Essence's retrieval step does not depend on this loop, since item representations are derived from content rather than interaction frequency: an item's likelihood of being recommended is a function of embedding similarity to the active cluster, independent of how many other users have interacted with it.

6.2 Deployment Positioning: A Candidate-Generation Method, Not a Ranking Replacement

The absolute recall values reported in Section 5 (approximately 0.01–0.02 under full-catalog evaluation) should be interpreted in light of how such a system would actually be deployed. Essence is not proposed as a replacement for a primary, CF-driven recommendation feed; CF baselines remain effective at surfacing broadly popular content efficiently, which is appropriate for a general homepage or default feed. Essence is better understood as a specialized retrieval path: a candidate-generation method suited to a dedicated discovery surface (for example, a "more like this" or deep-catalog-exploration interface) where the explicit goal is surfacing items dissimilar from what population-level signals would otherwise recommend. Framing Essence this way clarifies its evaluation criteria: it is not competing with CF on aggregate engagement metrics, but on its ability to recover items that CF-driven surfaces structurally underrepresent.

6.3 Privacy and Deployment Constraints

Because Essence consults no cross-user data at any stage, it is deployable in settings where collaborative filtering is not viable: privacy-regulated environments, on-device recommendation without server-side interaction logs, federated settings where data cannot leave the user's device, and cold-start contexts where a population-level interaction graph does not yet exist. This is a direct consequence of the architecture (Section 3) rather than an added feature, and follows from the same data-level distinction discussed in Section 2.2.

7 Limitations and Future Work

- **Scale.** Last.fm-1K evaluation uses 99 users. While Amazon Books replicates on 2,000 users, larger-scale evaluation on both datasets would strengthen generalisability claims. In particular, the Last.fm-1K long-tail comparison rests on few eligible long-tail hits (2 of 92 users per method), so that specific result is directional; the Amazon Books long-tail result (2,000 users) carries the statistical weight.

- K **sensitivity.** $K = 3$ is used throughout. An ablation over $K \in \{2, 3, 4, 5\}$ would characterise sensitivity to this hyperparameter across both domains.

- **Active cluster selection.** The recency-based heuristic is simple by design. A learned selection mechanism could improve session-aware accuracy without requiring cross-user training.
- **Filter bubble risk.** Peer-isolation removes the serendipity that cross-user signals can provide. Whether repeated centroid queries cause semantic narrowing is an open empirical question.
- **Embedding input curation.** Section 5.4 demonstrates that noisy embedding inputs degrade performance. The quality of what enters the embedding determines the quality of what Essence can find. Automated curation of embedding input text (filtering off-topic or irrelevant content before encoding) is a direct practical extension.
- **Audio-based embeddings.** Current embeddings derive from item metadata text only. For music, incorporating audio-based representations (capturing sonic texture, timbre, rhythm, and harmonic structure directly from audio signal) would more precisely model the latent acoustic fingerprint underlying individual taste. This represents the primary planned extension of Essence: replacing or augmenting text embeddings with audio embeddings and measuring whether long-tail recall improves when the embedding captures what the music actually sounds like rather than what it is called.
- **Cross-space embedding alignment.** Pass 2 results suggest that aligning review-text and metadata input distributions within the shared embedding space (via contrastive training or a learned projection layer) could unlock the full signal of user-authored representations while maintaining retrieval coherence.
- **Comparison against multi-interest baselines.** The architectural comparison in Section 2.2 identifies MIND and ComiRec as the closest prior systems. A direct empirical evaluation on a dataset where pre-trained MIND or ComiRec models are available would establish whether the long-tail advantage attributed to Essence's metadata-only embeddings persists against a well-tuned multi-interest baseline.

8 Conclusion

We presented Essence, a personal embedding cluster framework that generates recommendations by clustering a user's item history into K semantic centroids and retrieving items by similarity to the centroid closest to the user's recent activity, without consulting any cross-user data. Across two datasets (Last.fm-1K and Amazon Books), Essence outperforms a content-based average-embedding baseline, a non-personalized popularity baseline, and an item-based KNN collaborative filtering baseline on long-tail item recovery, under a chronological evaluation split and standard Recall@K. The non-personalized popularity baseline achieves exactly zero long-tail recall on both datasets, by construction. The item-based CF baseline recovers some long-tail items only on the denser Amazon Books dataset (comparable to the content baseline; overlapping CIs); on the sparser Last.fm-1K it recovers none under deterministic ranking, matching the popularity baseline, while on both it incurs a large cost to overall recall and remains below Essence. CF's long-tail recovery is thus density-dependent, whereas Essence leads on long-tail recall regardless of density. The long-tail comparison is statistically substantiated on Amazon Books (2,000 users; bootstrap 95% CIs, Table 5); on the 99-user Last.fm-1K it is directional, resting on long-tail hits for only 2 of 92 eligible users per method.

An ablation (Section 5.3) indicates that Essence's long-tail advantage depends on its recency-based active cluster selection mechanism specifically, not on K-means clustering alone: replacing recency-based selection with a static mean-embedding query degrades performance below the content baseline. This finding has been verified on Last.fm-1K only, and we have not yet confirmed whether it holds on Amazon Books.

The long-tail definition used here (items with exactly one training-set interaction) produced internally consistent results when applied independently to two datasets with different domains, scales, and interaction densities, without modification to the threshold or its derivation. We report this as a useful property of the metric for evaluating long-tail recovery, while noting that confirming generality beyond these two datasets would require testing on additional domains.

Limitations of the present evaluation, beyond those above, are discussed in Section 7, including dataset scale, sensitivity to the choice of K, and the increased recall degradation observed when taste profiles are built from noisy, user-authored text (Section 5.4) rather than structured item metadata.

References

[1] Li, C., Liu, Z., Wu, M., Xu, Y., Zhao, H., Huang, P., Kang, G., Chen, Q., Li, W., & Lee, D. L. (2019). *Multi-Interest Network with Dynamic Routing for Recommendation at Tmall.* CIKM 2019.

[2] Cen, Y., et al. (2020). *Controllable Multi-Interest Framework for Recommendation.* KDD 2020.

[3] Sun, F., et al. (2019). *BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer.* CIKM 2019.

[4] He, X., et al. (2017). *Neural Collaborative Filtering.* WWW 2017.

[5] Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems.* IEEE Computer.

[6] Hidasi, B., et al. (2016). *Session-Based Recommendations with Recurrent Neural Networks.* ICLR 2016.

[7] Wang, W., et al. (2020). *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers.* NeurIPS 2020.

[8] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP-IJCNLP 2019.

[9] Celma, O. (2010). *Music Recommendation and Discovery: The Long Tail, Long Fail, and Long Play in the Digital Music Space.* Springer.

[10] Hou, Y., et al. (2024). *Bridging Language and Items for Retrieval and Recommendation.* arXiv:2403.03952.

[11] Krichene, W., & Rendle, S. (2020). *On Sampled Metrics for Item Recommendation.* KDD 2020.